

PANDUAN LENGKAP DATABASE

Proyek DIGIDOC C | MySQL, phpMyAdmin, PHP, API, dan GitHub

Panduan praktik untuk memahami tipe data (VARCHAR, INT, dan lainnya), membuat database serta tabel, mengimpor data contoh, dan menghubungkan aplikasi ke MySQL.

Nama database proyek: `db_digidocc`

Berkas proyek utama: `database_schema.sql`, `dummy_data_import.sql`, `db.php`, dan folder `api`.

1. Peta konsep: database tidak sama dengan phpMyAdmin atau GitHub

MySQL adalah layanan/server yang menyimpan data. phpMyAdmin adalah alat berbasis web untuk mengelola MySQL. Database adalah wadah data di dalam MySQL; tabel adalah struktur data di dalam database. Aplikasi PHP mengirim query ke MySQL. Git/GitHub mengelola dan menyimpan file kode, bukan database MySQL yang sedang berjalan.

Browser (HTML/JavaScript) -> permintaan HTTP -> PHP API -> `db.php` (PDO) -> MySQL -> `db_digidocc` -> tabel

Git di VS Code -> commit/push file `.php`, `.js`, `.sql` -> GitHub

- `database_schema.sql` berisi instruksi SQL untuk membuat database, tabel, relasi, index, dan beberapa data awal.
- `dummy_data_import.sql` berisi instruksi untuk menambah data contoh ke tabel.
- `db.php` adalah file konfigurasi koneksi database yang dipakai API PHP saat sistem berjalan.
- phpMyAdmin bukan file koneksi dan bukan database itu sendiri; phpMyAdmin hanya alat untuk melihat atau mengelola server MySQL.

2. Istilah tipe data yang benar

Ejaan yang benar adalah VARCHAR dan INT (bukan “varchar” atau “imt”). Tipe data menentukan jenis nilai yang boleh disimpan dalam sebuah kolom.

Tipe data yang digunakan pada skema Digidocc

Makna umum tipe-tipe data yang benar-benar dipakai dalam schema proyek:

Kolom	Tipe data	Aturan	Arti / fungsi
id	INT	Bilangan bulat	Dipakai untuk identitas numerik seperti id, dock_id, dan user_id.
BIGINT	BIGINT	Bilangan bulat berkapasitas lebih besar	Dipakai untuk id log aktivitas yang bisa terus bertambah.

VARCHAR(n)	VARCHAR dengan batas n karakter	Teks dengan panjang bervariasi	Contoh VARCHAR(50) untuk username; panjang aktual dapat lebih pendek dari batas.
CHAR(n)	CHAR dengan panjang tetap	Teks pendek berukuran tetap	CHAR(3) cocok untuk kode mata uang tiga huruf seperti IDR.
DECIMAL(p,s)	Angka desimal presisi tetap	p = jumlah digit total; s = digit di belakang koma	Contoh DECIMAL(15,2) untuk nominal uang; hindari FLOAT untuk nominal yang harus presisi.
DATE	Tanggal	Tahun-bulan-hari	Contoh 2026-10-05; digunakan oleh schedule_date.
DATETIME	Tanggal dan waktu	Tanggal beserta jam-menit-detik	Digunakan untuk waktu sandar kapal: stay_start dan stay_end.
TIMESTAMP	Waktu pencatatan	Dapat diberi default waktu saat data dibuat/diubah	created_at mencatat waktu pembuatan; updated_at dapat berubah otomatis.
ENUM(...)	Pilihan teks yang dibatasi	Nilai harus salah satu opsi yang didefinisikan	Contoh status kapal hanya waiting, in_progress, completed, atau cancelled.
TEXT	Teks panjang	Untuk isi yang lebih panjang	Dipakai sebagai catatan jadwal pada notes.
JSON	Objek/data terstruktur JSON	Menyimpan pasangan kunci-nilai	Dipakai untuk detail tambahan pada activity_logs.

p dan s pada DECIMAL

DECIMAL(10,2) berarti total maksimal 10 digit, dengan 2 digit pecahan. DECIMAL(15,2) berarti maksimal 15 digit total dan 2 digit pecahan. Untuk ukuran kapal/dermaga dan uang pada skema ini, nilai disimpan secara desimal.

Aturan kolom yang sering terlihat

- NOT NULL: nilai wajib diisi, tidak boleh NULL (kosong/tidak diketahui). Ini berbeda dengan teks kosong (string kosong).
- NULL: nilai boleh belum tersedia atau tidak diketahui.
- PRIMARY KEY: identitas unik untuk setiap baris, biasanya id.
- AUTO_INCREMENT: MySQL mengisi id berikutnya secara otomatis.
- UNIQUE: nilai tidak boleh sama dengan baris lain, misalnya username dan kode dermaga.
- DEFAULT: nilai yang digunakan bila tidak diberikan, misalnya status default waiting.

- FOREIGN KEY: nilai rujukan ke baris di tabel lain agar relasi tetap valid.

3. Langkah persiapan XAMPP dan phpMyAdmin

1. Buka XAMPP Control Panel. Jalankan MySQL. Jalankan Apache untuk membuka phpMyAdmin dan menjalankan endpoint PHP melalui web.
2. Buka browser dan kunjungi <http://localhost/phpmyadmin>.

Jika XAMPP tidak digunakan, jalankan layanan MySQL dan web server PHP dari instalasi yang memang digunakan. Nama pengguna/kata sandi MySQL dapat berbeda.

4. Cara membuat database dan tabel dengan file SQL proyek (disarankan)

Cara ini paling konsisten karena struktur yang dibuat dari skema akan sesuai dengan kode proyek.

3. Di phpMyAdmin, buka tab Databases. Buat database dengan nama `db_digidocc` dan collation `utf8mb4_unicode_ci` jika database tersebut belum ada.
4. Klik database `db_digidocc` pada panel kiri. Buka tab Import.
5. Pilih file `database_schema.sql` yang berada di folder utama proyek Digidocc. Pastikan file yang dipilih adalah schema, bukan file dummy.
6. Jalankan Import/Go dan tunggu pesan keberhasilan. Buka database di panel kiri untuk memastikan tabel dibuat.
7. Pilih tab Import sekali lagi, pilih `dummy_data_import.sql`, lalu jalankan Import/Go untuk menambahkan data contoh.
8. Klik beberapa tabel lalu pilih tab Browse untuk memeriksa datanya.

Catatan penting: `database_schema.sql` membuat database `db_digidocc` jika belum ada, memilihnya dengan `USE`, membuat tabel, memasukkan beberapa data dan akun admin, serta membuat index. Jangan impor ulang schema ke database yang telah disiapkan tanpa backup/periksa dahulu: perintah `CREATE INDEX` di schema dapat gagal jika index dengan nama sama sudah ada.

Jika hanya ingin membuat database baru dan tabelnya secara otomatis dari file, schema dapat diimpor dari halaman database/server karena berisi `CREATE DATABASE` dan `USE`. Cara paling mudah dipahami adalah buat/pilih `db_digidocc` terlebih dahulu, kemudian impor file schema.

5. Cara membuat lewat tab SQL (contoh untuk memahami proses)

Di phpMyAdmin, pilih database `db_digidocc`, buka tab SQL, tempel perintah, lalu klik Go. Contoh berikut membuat satu tabel `users` versi ringkas. Jika seluruh schema proyek sudah diimpor, jangan jalankan contoh `CREATE TABLE` ini lagi pada tabel yang sama.

```
CREATE TABLE users_example (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  id_number VARCHAR(20) NOT NULL,
```

```

user_type ENUM('admin','employee','customer') NOT NULL DEFAULT 'employee',

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);

```

Dalam proyek yang sebenarnya, nama tabelnya adalah `users` (bukan `users_example`). Contoh dibuat memakai nama berbeda agar tidak menabrak tabel sebenarnya. File `database_schema.sql` berisi definisi lengkap dan relasi semua tabel; lebih aman memakai file tersebut daripada mengetik ulang satu demi satu.

6. Daftar tabel dan arti setiap kolom

Berikut struktur yang didefinisikan di `database_schema.sql` proyek Digidocc.

users — akun pengguna

Menyimpan akun untuk login, jenis pengguna, serta waktu pembuatan/perubahan akun.

Kolom	Tipe data	Aturan	Arti / fungsi
id	INT	PK, AUTO_INCREMENT	Identitas pengguna yang dibuat otomatis.
username	VARCHAR(50)	NOT NULL, UNIQUE	Nama login; wajib dan tidak boleh kembar.
password_hash	VARCHAR(255)	NOT NULL	Hash kata sandi, bukan kata sandi polos.
id_number	VARCHAR(20)	NOT NULL	Nomor/identitas pengguna, misalnya ADM atau EMP1.
user_type	ENUM(admin, employee, customer)	NOT NULL, DEFAULT employee	Jenis akun yang dibatasi ke tiga opsi.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Waktu akun dibuat.
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Waktu perubahan terakhir; diperbarui otomatis.

docks — data master dermaga

Menyimpan kode, nama, lokasi, ukuran, dan status setiap dermaga.

Kolom	Tipe data	Aturan	Arti / fungsi
id	INT	PK, AUTO_INCREMENT	Identitas dermaga.
code	VARCHAR(20)	NOT NULL, UNIQUE	Kode dermaga unik seperti IR, SMG, atau SBY.
name	VARCHAR(100)	NOT NULL	Nama dermaga.
location	VARCHAR(100)	NOT NULL	Lokasi dermaga.

dock_length	DECIMAL(10,2)	NOT NULL	Panjang dermaga; satuan yang digunakan aplikasi adalah meter.
dock_width	DECIMAL(10,2)	NOT NULL	Lebar dermaga; satuan yang digunakan aplikasi adalah meter.
status	ENUM(active, maintenance, inactive)	NOT NULL, DEFAULT active	Status operasional dermaga.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Waktu data dibuat.
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Waktu data terakhir diperbarui.

ships — data kapal dan masa sandar

Menyimpan kapal, ukuran/kapasitas, dermaga, waktu sandar, status, dan pembuat data.

Kolom	Tipe data	Aturan	Arti / fungsi
id	INT	PK, AUTO_INCREMENT	Identitas data kapal.
dock_id	INT	NOT NULL, FK ke docks.id	Dermaga tempat kapal tercatat.
ship_name	VARCHAR(100)	NOT NULL	Nama kapal.
loa	DECIMAL(10,2)	NOT NULL	Length Overall: panjang kapal keseluruhan; umumnya meter.
beam	DECIMAL(10,2)	NOT NULL	Lebar kapal; umumnya meter.
draft	DECIMAL(10,2)	NOT NULL	Sarat/draft kapal; kedalaman kapal di bawah permukaan air; umumnya meter.
gt	DECIMAL(10,2)	NOT NULL	Gross Tonnage, ukuran kapasitas volume internal kapal; bukan berat ton sederhana.
dwt	DECIMAL(10,2)	NOT NULL	Deadweight Tonnage, kapasitas bobot total yang dapat dibawa kapal.
stay_start	DATETIME	NOT NULL	Tanggal dan waktu mulai sandar.
stay_end	DATETIME	NOT NULL	Tanggal dan waktu selesai sandar; harus

			tidak lebih awal dari stay_start.
status	ENUM(waiting, in_progress, completed, cancelled)	NOT NULL, DEFAULT waiting	Status proses kapal.
created_by	INT	NULL, FK ke users.id	Pengguna yang membuat data; boleh kosong.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Waktu data kapal dibuat.
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Waktu data kapal diperbarui.

ship_schedules — jadwal kapal

Menyimpan rincian tanggal jadwal dan catatan, terkait sebuah kapal dan dermaga.

Kolom	Tipe data	Aturan	Arti / fungsi
id	INT	PK, AUTO_INCREMENT	Identitas baris jadwal.
ship_id	INT	NOT NULL, FK ke ships.id	Kapal yang dijadwalkan.
dock_id	INT	NOT NULL, FK ke docks.id	Dermaga yang digunakan pada jadwal.
schedule_date	DATE	NOT NULL	Tanggal jadwal.
notes	TEXT	NULL	Catatan tambahan; boleh tidak diisi.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Waktu jadwal dibuat.

Catatan: tabel jadwal sudah ada dalam schema, tetapi file dummy_data_import.sql yang disiapkan tidak mengisi ship_schedules. Data kapal contoh berada di ships.

vessel_sales — data vessel dan nilai anggaran

Menyimpan vessel/jenis penjualan dan nilai budget utama.

Kolom	Tipe data	Aturan	Arti / fungsi
id	INT	PK, AUTO_INCREMENT	Identitas data vessel.
vessel_name	VARCHAR(100)	NOT NULL	Nama vessel atau unit.
sales_type	ENUM(Navy Vessels (KRI), Commercial Vessel, Non-Vessel)	NOT NULL	Kategori penjualan yang diizinkan.

budget	DECIMAL(15,2)	NULL	Budget utama vessel; dapat belum ditentukan.
currency	CHAR(3)	NOT NULL, DEFAULT IDR	Kode mata uang tiga huruf, misalnya IDR.
created_by	INT	NULL, FK ke users.id	Pengguna pembuat data, boleh kosong.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Waktu data dibuat.
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Waktu data terakhir diperbarui.

sales_budget — rincian anggaran

Menyimpan rincian nilai budget per kategori untuk sebuah vessel.

Kolom	Tipe data	Aturan	Arti / fungsi
id	INT	PK, AUTO_INCREMENT	Identitas rincian budget.
vessel_id	INT	NOT NULL, FK ke vessel_sales.id	Vessel pemilik rincian budget.
category	VARCHAR(100)	NOT NULL	Nama kategori biaya/anggaran seperti Maintenance atau Operations.
amount	DECIMAL(15,2)	NOT NULL	Jumlah anggaran untuk kategori.
currency	CHAR(3)	NOT NULL, DEFAULT IDR	Kode mata uang.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Waktu rincian dibuat.

activity_logs — catatan aktivitas

Mencatat aksi aplikasi dan detailnya untuk kebutuhan riwayat/audit.

Kolom	Tipe data	Aturan	Arti / fungsi
id	BIGINT	PK, AUTO_INCREMENT	Identitas log berkapasitas lebih besar.
user_id	INT	NULL, FK ke users.id	Pengguna pelaku aktivitas; boleh kosong.
action	VARCHAR(100)	NOT NULL	Nama aksi, misalnya login atau create_ship.

module	VARCHAR(100)	NOT NULL	Bagian aplikasi terkait, misalnya auth, dock, atau sales.
details	JSON	NULL	Detail tambahan berformat JSON.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Waktu aktivitas dicatat.

7. Memahami relasi antar tabel

users (1) ---- (banyak) ships melalui ships.created_by
docks (1) ---- (banyak) ships melalui ships.dock_id
ships (1) ---- (banyak) ship_schedules melalui ship_schedules.ship_id
docks (1) ---- (banyak) ship_schedules melalui ship_schedules.dock_id
users (1) ---- (banyak) vessel_sales melalui vessel_sales.created_by
vessel_sales (1) - (banyak) sales_budget melalui sales_budget.vessel_id
users (1) ---- (banyak) activity_logs melalui activity_logs.user_id

FK berarti foreign key/kunci asing: kolom pada satu tabel yang merujuk ke primary key tabel lain. Misalnya ships.dock_id menyimpan nilai docks.id. Saat aplikasi menyimpan kapal, dermaga yang dirujuk harus sudah ada.

Aturan penghapusan yang tertulis pada schema

- ON DELETE RESTRICT pada relasi kapal/jadwal ke dermaga mencegah dermaga yang masih dipakai dihapus.
- ON DELETE SET NULL pada created_by/user_id mempertahankan catatan kapal, vessel, atau log; kolom pengguna menjadi NULL jika user dihapus.
- ON DELETE CASCADE pada ship_schedules terhadap ships berarti jadwal terkait ikut terhapus jika kapal dihapus.
- ON DELETE CASCADE pada sales_budget terhadap vessel_sales berarti rincian budget ikut terhapus jika vessel induk dihapus.

8. Bagaimana koneksi sistem dibuat? (bagian terpenting)

Pada proyek ini koneksi disiapkan dalam file baru di folder utama bernama db.php. PHP API memanggil file ini; file HTML/JavaScript tidak menyimpan kata sandi database dan tidak berbicara langsung dengan MySQL.

Isi koneksi proyek saat ini

```
<?php
$host = 'localhost';
$dbname = 'db_digidocc';
$dbuser = 'root';
$dbpass = '';
```

```
$pdo = new PDO(
    "mysql:host={$host};dbname={$dbname};charset=utf8mb4",
    $dbuser,
    $dbpass,
    [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
     PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC]
);
?>
```

Arti konfigurasi

- \$host adalah alamat server MySQL; localhost berarti server berada di komputer yang sama.
- \$dbname adalah database yang dipilih; pada proyek ini db_digidocc.
- \$dbuser dan \$dbpass adalah kredensial akun MySQL, bukan akun login pengguna aplikasi.
- PDO adalah lapisan akses database PHP. Objek koneksinya disimpan di variabel \$pdo.
- charset=utf8mb4 membantu menyimpan teks Unicode dengan benar.

Endpoint API memuat koneksi dengan include

```
require_once __DIR__ . '/../db.php';
```

Contohnya baris pertama setelah pembuka PHP di api/ships.php, api/login.php, api/register.php, api/docks.php, api/vessels.php, api/users.php, dan api/seed_admin.php. Sesudah db.php dimuat, endpoint bisa menggunakan variabel \$pdo untuk menjalankan query.

Contoh query dalam api/ships.php

```
$stmt = $pdo->query(
    SELECT s.*, d.code AS dock_code, d.name AS dock_name
    FROM ships s
    LEFT JOIN docks d ON d.id = s.dock_id
    ORDER BY s.stay_start DESC
);
```

Query tersebut membaca data ships dan menggabungkan kode/nama dock. Endpoint mengembalikan hasil sebagai JSON ke browser.

Contoh pengiriman permintaan dari JavaScript

```
const response = await fetch('./api/ships.php');
const result = await response.json();
```

Pada halaman login, login.js memanggil api/login.php, pendaftaran memanggil api/register.php, dan daftar pengguna memanggil api/users.php. Untuk data kapal, script.js, semarang.js, dan sby.js memanggil api/ships.php. Maka alur lengkapnya adalah browser -> endpoint PHP -> db.php/PDO -> MySQL -> hasil JSON -> browser.

9. Menjalankan dan menguji sistem lokal

9. Pastikan database db_digidocc sudah dibuat dan file schema sudah diimpor.
10. Di XAMPP, nyalakan Apache dan MySQL.
11. Pastikan folder proyek berada di document root Apache, biasanya C:\xampp\htdocs\digidocc.
12. Buka aplikasi melalui http://localhost/digidocc/ (atau alamat sesuai nama folder). Jangan hanya membuka file HTML dengan file:/// jika ingin endpoint PHP berjalan.
13. Tes GET sederhana dengan membuka http://localhost/digidocc/api/ships.php. Jika koneksi dan API berjalan, respons berupa JSON dengan success dan data kapal (atau array kosong bila belum ada kapal).
14. Coba halaman login atau data kapal. Periksa respons API jika muncul pesan kesalahan koneksi/database.

Jika proyek diletakkan pada alamat/folder berbeda, ubah URL sesuai lokasinya. Path API relatif seperti ./api bekerja ketika halaman dibuka melalui web server pada folder proyek.

10. Penyimpanan database vs localStorage

localStorage adalah penyimpanan di browser untuk situs tersebut; MySQL adalah database server. Keduanya berbeda dan tidak otomatis saling menyalin semua data. Pada proyek ini beberapa JavaScript masih membaca/menyimpan localStorage atau menyediakan fallback lokal, sementara sejumlah fitur memanggil API PHP. Karena itu jangan mengasumsikan setiap data lama pada localStorage otomatis sudah masuk ke MySQL. Mengimpor SQL ke phpMyAdmin tidak menghapus localStorage.

11. Git/VCS dan GitHub: apa yang masuk repository?

Source Control di VS Code menampilkan perubahan file yang diketahui Git. File database_schema.sql, dummy_data_import.sql, db.php, dan API PHP dapat menjadi bagian repository dan dikirim ke GitHub melalui commit/push. Database server MySQL lokal tidak terkirim otomatis.

- File schema adalah resep untuk membangun tabel di MySQL lain.
- File dummy adalah resep untuk mengisi data uji.
- db.php adalah kode koneksi aplikasi; jangan mengunggah password rahasia/server produksi ke repository publik.
- Pada hosting, buat database dan user MySQL dari panel hosting, lalu atur kredensial koneksi produksi dengan aman.

12. Masalah umum dan pemeriksaan cepat

Pesan/gejala dan solusinya

Gunakan checklist ini sebelum mengubah kode atau menghapus database.

Kolom	Tipe data	Aturan	Arti / fungsi
Unknown database db_digidocc	Database belum dibuat atau namanya berbeda	Periksa daftar database di phpMyAdmin dan	

		nama \$dbname di db.php.	
Access denied for user	Nama pengguna/kata sandi atau izin salah	Periksa \$dbuser, \$dbpass, dan hak akses MySQL.	
Connection refused / MySQL server has gone away	Layanan MySQL mati atau host/port salah	Nyalakan MySQL di XAMPP dan pastikan konfigurasi server benar.	
404 pada api/ships.php	Path/folder web server salah atau Apache tidak berjalan	Pastikan folder proyek ada di htdocs dan URL memakai localhost/nama-folder.	
File PHP tampil sebagai teks / tidak berjalan	Halaman dibuka tanpa web server PHP	Jalankan Apache dan buka aplikasi melalui http://localhost/.	
Tabel tidak ditemukan	Schema belum diimpor ke database yang dipilih	Periksa nama database di sidebar phpMyAdmin dan jalankan schema untuk database baru.	
Index already exists saat impor ulang schema	Schema sudah pernah dijalankan	Jangan impor ulang tanpa backup; gunakan database kosong baru atau sesuaikan proses migrasi dengan hati-hati.	

13. Checklist singkat dari awal sampai berhasil

- MySQL aktif di XAMPP.
- Database db_digidocc tersedia di phpMyAdmin.
- database_schema.sql sudah diimpor dan tabel terlihat.
- dummy_data_import.sql diimpor jika perlu data contoh.
- db.php menunjuk ke host, database, user, dan password MySQL lokal yang benar.
- Semua API membutuhkan koneksi dengan require_once ke db.php.
- Aplikasi dibuka melalui Apache/http://localhost, bukan file://.
- Endpoint API mengembalikan JSON tanpa error koneksi.

Ringkasan akhir

Yang membuat tabel adalah instruksi SQL dalam database_schema.sql yang dijalankan oleh MySQL (dapat dikelola melalui phpMyAdmin). Yang menyambungkan sistem PHP ke MySQL adalah db.php. File api/*.php memuat db.php dan menjalankan query melalui \$pdo. JavaScript memanggil API, sedangkan GitHub menyimpan file proyek—bukan database MySQL aktif.

Panduan ini mengikuti struktur file dan database Digidocc. Detail koneksi MySQL lokal dapat berbeda di komputer masing-masing.

